

Who says what, and when

One page per person. Print your own page and hold it.

Clock	Speaker	Part	Slides	The one thing they must land
0:00 – 2:00	Adyasha	Introduction	1 – 4	It is a <i>proof</i> problem, not a translation problem — and in India it is a supervisory event
2:00 – 4:15	Swayam	Architecture	5 – 7	A deterministic pipeline with the AI on a short leash, and a reciprocal loop at its centre
4:15 – 7:00	Soumyajit	Demo & working	8 + live demo	Our own AI got a rule wrong and our own harness caught it — here is the counterexample
7:00 – 9:30	Tamanna	Business model	9 – 12	The money is already committed; assurance is the gate, and nobody sells it
9:30 – 10:00	Buffer. If you reach 9:30 with everything said, you have run it correctly. Slide 13 (References) stays on screen for Q&A.			

Rules for all four of you

- **Do not open with pleasantries.** No “good morning, we are Team StudyEdge, today we will present...” That is thirty seconds of a six-hundred-second budget. Adyasha opens on the problem; the title slide carries the names.
- **Never claim we invented differential testing.** IBM, AWS and Mechanical Orchard all ship a version. Saying so first is what makes the rest of the pitch credible. The claims that survive scrutiny are the regulatory third reference, the per-rule verdict, and testing our own tests.
- **Never say “proof of correctness” or “we verified it.”** Say “*evidence of equivalence over the executed input set.*” A judge with a formal-methods background will pounce on the stronger word, and we lose more from that than we gain.
- **Banned numbers:** “95% of ATM swipes run on COBOL” and “800 billion lines.” Both are contested. Use *92 of the top 100 banks* and *over 40% of banking systems*.
- **If you do not know, say so and name who does.** “That is Swayam’s part — Swayam?” is a strong answer. Guessing in front of a Cognizant panel is not.
- **Watch the clock, not the slides.** If the person before you overran, cut from your own *cut-first* list (bottom of your page) rather than compressing the ending.

Q&A — who takes which question

Question	Primary	Backup
"IBM watsonx / AWS Blu Age already validates this. What is new?" — <i>the one that will definitely come</i>	Soumyajit	Swayam on the technical differences
"There is a paper from last week doing this — Locksmith, arXiv 2607.28271." — <i>the ambush</i>	Soumyajit	—
"Your extraction is an LLM. How do you know the rules are right?"	Swayam	Soumyajit — the refuted rule <i>is</i> the answer
"GnuCOBOL is not IBM Enterprise COBOL."	Swayam	—
"Interest accrual is a toy problem."	Swayam	Tamanna on scope as a decision
"Everything passed. What have you actually proved?"	Soumyajit	—
"Who buys this? Why not just call Cognizant?"	Tamanna	Adyasha on the stakeholder map
Market size, ROI, payback, pricing	Tamanna	—
"What is the hardest part, and what have you not solved?"	Soumyajit	Answer straight. Hedging costs more than the admission.

The single most likely way this goes wrong: a judge who works on mainframe modernisation hears the pitch as unaware of IBM and AWS. Adyasha and Swayam should both name those vendors *before* any claim of novelty is made. Naming them ourselves converts the objection into evidence of research depth. Full answers live in Part 5 of the R2 dossier.

Adyasha

CLOCK

0:00 → 2:00

SLIDES

1, 2, 3, 4

HAND OFF TO

Swayam

YOUR JOB

Make the room believe the problem is real, expensive and specifically Indian — and that the unsolved half is *proof*, not translation. Everything the other three say depends on the room accepting this. You are not describing a project; you are setting up a gap that Swayam then fills.

0:00 Slide 1 — Title. Do not read it out.

“India’s banks run on code that nobody can safely change. In the next ten minutes we want to convince you that is a proof problem, not a programming problem.”

Then move on. The names are on the slide; you do not need to introduce four people out loud.

0:15 Slide 2 — The problem. Lead with the four numbers.

92 of the world’s top 100 banks run on IBM mainframes. Over 40% of banking systems still run COBOL written decades ago. BCG puts the Indian core-banking upgrade bill at about **\$1 billion over five to ten years**. And the fourth number is **zero** — the number of specifications stating what those programs are supposed to do.

Then the consequence, which is the point of the slide:

“When an Indian bank guesses wrong, it is not a bug report, it is a supervisory event. TSB in the UK, 2018: a failed cutover, 1.9 million of 5.2 million customers locked out, a £48.65 million fine. HDFC Bank, 2020: after repeated outages the RBI froze every Digital 2.0 launch and all new credit-card issuance — the card freeze lifted only in August 2021, the digital freeze ran into 2022.”

Land this: refusing to touch the system is the *rational* decision under the bank’s own risk framework. That is why the backlog is measured in decades.

1:00 Slide 3 — Why it is unsolved. The most important slide you own.

“Translation is solved. Transpilers and LLMs do it, and AWS, IBM and TCS all ship it. The chain breaks at the last step. No specification was ever written, so the rules live only inside the code — and asking an AI to explain them looks excellent while a meaningful share of the output is wrong. The best published COBOL rule extractor reaches 62% precision. A 2026 system extracted 91.2% of business rules and reached under 20% behavioural equivalence. Extraction works. Equivalence does not follow from it.”

Close on the line the whole pitch hangs from: *“Verification has to be the product.”*

1:40 Slide 4 — Target users and stakeholders. Twenty seconds, no more.

Bank CIO, risk and internal audit, the modernisation vendors, and the supervisor. Do not read the table — deliver the one line under it:

“Nobody on this list is short of a translator. Everybody on this list is short of proof. Our buyer is the person who has to sign the cutover memo, and today they have nothing to attach to it.”

HAND OFF

“Swayam will show you what we built to give them something to attach.”

Cut first if you are over: the stakeholder table on slide 4 — drop straight to the “nobody is short of a translator” line. **Never cut:** the HDFC/TSB consequence, and “verification has to be the product.”

Swayam

CLOCK

2:00 → 4:15

SLIDES

5, 6, 7

HAND OFF TO

Soumyajit

YOUR JOB

Make the system feel *disciplined*. The impression to leave is that the AI is on a short leash and the design refuses what it cannot verify soundly. A judge who suspects this is an LLM wrapper will discount everything Soumyajit shows next — your two minutes are what stops that.

2:00 Slide 5 — Our solution. Walk the five stages, roughly ten seconds each.

0 • Intake gate. Programs that read the system clock, hit a database, or depend on CICS terminal state are *refused, with a reason*. Differential testing needs determinism — same input, same output, every run. *“We would rather refuse a program than issue a confident verdict we cannot support.”*

1–2 • Parse and slice. The ProLeap parser builds the AST; the program is cut into rule-sized slices.

3 • Extract rules. One bounded LLM call per slice — never over the whole file — and line citations are written by the pipeline, not the model, so it cannot invent a reference to lines the slice does not contain.

4 • Rule-directed input generation. The rules tell us where the boundaries are, so the generator emits ₹99,999 / ₹1,00,000 / ₹1,00,001, accruals landing on exactly ₹0.50, and 29 February.

5 • Execute and compare, three ways. Legacy COBOL, the untrusted Java, and the regulation.

Say the leash out loud: *one* bounded LLM call site in the entire pipeline. Everything else is deterministic.

2:50 Slide 6 — How it works. Point at the dashed loop; that is the contribution.

“This dashed path is the reciprocal loop, and it is the part that is genuinely ours. The extracted rules pick which inputs get generated; the execution run then judges the rules. Extraction is not a deliverable sitting next to the verification — it is inside it, and the verification is what catches it lying.”

Then introduce the third reference, because Soumyajit will demo it:

“And there is a third input to the comparison: about sixty lines of executable model encoding what the RBI Master Direction actually prescribes, clause by clause — savings interest on a daily product basis, rupee-deposit interest rounded to the nearest rupee. Every other tool compares two things. We compare three.”

3:35 Slide 7 — Technical details. One line each, then the closer.

GnuCOBOL — free and open, runs on a laptop, 19 dialects including an IBM-strict mode aligned to Enterprise COBOL 6.3, passes 9,700 of 9,748 NIST conformance tests. **ProLeap** — ANTLR4 grammar, AST plus a semantic graph with data and control flow. **gcov** for statement coverage. **Delta debugging** — shrinks 73 failing inputs down to one minimal counterexample.

Close on portability, because it pre-empts a Q&A question:

“No mainframe access is required anywhere in this. The methodology is compiler-agnostic — swap GnuCOBOL for the bank’s own compiler inside the bank, and the harness is unchanged.”

HAND OFF

“That is the design. Soumyajit is going to run it, and show you the part where it catches us being wrong.”

Cut first if you are over: on slide 7, keep GnuCOBOL and delta debugging, drop ProLeap and gcov detail. **Never cut:** the intake gate refusing programs, the one-LLM-call leash, and the reciprocal loop.

Soumyajit

CLOCK

4:15 → 7:00

SLIDES

8 + live demo

HAND OFF TO

Tamanna

YOUR JOB

You have the middle of the presentation, and the moment the room either believes us or does not. Do not explain the demo — *perform* it, in seven beats. Beat 5 is when the room understands the thesis; beat 6 is when a banker starts caring. Protect those two above everything else on this page.

4:15 Slide 8 — What makes it different. Thirty seconds, then get off it.

Three claims, fast: we check the legacy against the regulator; a verdict per business rule, not per test case; we test our own tests. Then disarm the prior-art objection immediately:

“The differential mechanism itself is prior art. IBM, AWS and Mechanical Orchard all ship a version, and a paper published five days before we compiled this does too. We are not claiming it. These three rows are ours.”

4:45 Demo · beats 1–3 — set-up. Move briskly; nothing here is the point yet.

1 • The COBOL. A savings-accrual program, about four hundred lines, no comments. *“This is the specification. There is no other one.”*

2 • The rules. Twenty extracted, each with its line numbers. *“That took eleven seconds. Roughly a quarter of it is probably wrong — which is exactly why this is not the product.”*

3 • The corpus. Boundaries derived from the rules themselves — 99,999, 1,00,000, 1,00,001; accruals landing on exactly ₹0.50; 29 February. *“The rules tell us where to look.”*

5:35 Demo · beat 4 — the divergence. Delta-debugged to one line.

“Balance one lakh, twelve days at 3.5%. The legacy says ₹115. The rewrite says ₹116. The Java uses HALF_UP; the COBOL truncates at the PIC boundary. Across a five-hundred-crore book that is not a bug report — it is a rounding drift the auditor finds first.”

6:05 Demo · beat 5 — the refuted rule. Slow down. Pause after it.

“Rule eleven says dormant accounts drop to the base rate after twenty-four months. We tested twenty-three months and twenty-five months. Identical output, both implementations. No such boundary exists in the program — our own extraction invented it, and our own harness caught it. An engineer reading that document would have built on it.”

This is the beat that proves the thesis: an AI system catching its own AI lying, with a reproducible counterexample. Give it a beat of silence before moving on.

6:25 Demo · beat 6 — the compliance finding. The one a banker reacts to.

“Here both implementations agree with each other, and both round interest to two decimal places. The RBI Master Direction says interest on a rupee deposit is rounded to the nearest rupee. So this is not a migration defect. This has been running for forty years, and a two-way diff cannot see it at all.”

6:45 Demo · beat 7 — the coverage table. Close on what we could *not* prove.

31 proven, 11 unproven, 5 refuted, 94% mutation kill rate.

“And these eleven we could not prove. We are telling you which ones. That is the difference between evidence and reassurance.”

HAND OFF

“Tamanna will tell you why a bank pays for this.”

If the demo breaks, do not debug in front of the panel. Switch to the rendered screenshots and narrate the same seven beats in the same order. Have that tab open before you start.

If asked whether the numbers are measured: say plainly that they are the illustrative output shape for the accrual program, not a completed production run. Volunteer it rather than let it be discovered.

Cut first if you are over: beat 3 (the corpus) — fold it into one clause of beat 2 — and shorten beat 7 to the four numbers plus the last sentence. **Never cut:** beats 5 and 6.

Tamanna

CLOCK
7:00 → 9:30

SLIDES
9, 10, 11, 12

HAND OFF TO
Close & Q&A

YOUR JOB

Convert a technical claim into a purchase decision, and close the presentation. Your central argument is that *the money is already committed* — we are not asking anyone to find a new budget, we are unblocking one that is approved and stalled. You also deliver the final line, so the last thing the panel hears is ours.

7:00 Slide 9 — Market potential. Four numbers, then the argument.

~\$1 billion of Indian core-banking upgrade spend over five to ten years (BCG). 92 of the 100 largest banks on mainframes. Over 40% of banking systems still on COBOL. Indian banks spending 5% of revenue on IT against 7–9% globally.

Then the sentence that makes it a business, not a statistic:

“Every rupee of that is gated by one step. Modernisation budgets do get approved. What stalls them is the assurance step between “the new code exists” and “we are willing to run it on real money” — and that step has no product category yet. Today it is absorbed into consulting hours and schedule risk.”

Then concede the competition, and turn it:

“The category is forming around us. AWS Transform for mainframe went generally available in May 2025. IBM watsonx Code Assistant for Z ships a validation assistant. Mechanical Orchard ships a generate-then-validate loop. All of them generate. None of them sells the proof as the product.”

7:45 Slide 10 — Feasibility and scope. Thirty seconds. Frame scope as a decision.

In scope: pure-computation retail accrual, third-party COBOL including AWS’s own CardDemo sample. Out of scope and detected, not ignored: CICS screens, JCL orchestration, database-coupled programs, anything non-deterministic — refused at intake with a reason.

“Narrow scope is a design decision, not a limitation we are hoping nobody notices.”

8:15 Slide 11 — Business plan, WHY / HOW / WHAT. Spend your time on WHAT.

Skim WHY and HOW — the panel has just heard both. Deliver the value proposition as four things the bank walks away holding: **a specification that never existed**, each rule traced to the lines that produce it; **a permanent regression suite**, reusable on every future change to that program; **an equivalence claim someone can sign**, verified, unproven or refuted per rule; and **compliance findings**, because the legacy gets checked against the regulator too.

Then the line that de-risks the whole purchase:

“And there is value even if the migration is cancelled. The specification and the test suite are retained regardless — the bank ends up with documentation for a system it previously could not touch.”

8:50 Slide 12 — Investments, returns, timelines. Fast, then close.

Investments: licence cost nil — GnuCOBOL, ProLeap, gcov, the NIST suite and AWS CardDemo are all open, no vendor contract. Prototype: four engineers, twelve weeks, commodity laptops. The only variable cost is bounded inference.

Returns: comparable automation cuts rule-extraction time by 80% and manual effort by over 90%. And the counterfactual is on the slide: *“TSB, 2018 — about £400 million all-in, the CEO resigned, and the regulator’s finding was about risk management and assurance, not about programming. The downside is two orders of magnitude above what the platform costs.”*

Timelines: working differential harness in weeks one to two — that already exists. The reciprocal loop and per-rule verdicts, weeks eight to ten, which is the contribution. Bank pilot, months four to nine, inside the bank’s own compiler environment.

If you cite the week numbers, add: *“the calendar mapping is our own sizing, and we have marked it as an assumption rather than a finding.”* That honesty is a scoring asset with this panel, not a weakness.

Close the presentation on this — then stop talking.

“Of forty-seven rules we extracted, thirty-one are proven, sixteen we could not prove — and here are the sixteen. Reporting what we did not prove is what separates evidence from reassurance.”

Cut first if you are over: slide 10 down to the single “narrow scope is a design decision” line, and skip WHY/HOW on slide 11 entirely.
Never cut: “the money is already committed / assurance is the gate”, the TSB counterfactual, and the closing line.